Recovery Debt — Master Overview

One document. Everything we've planned. The full strategy for beating WHOOP Coach.

Date: 2026-04-27

---

1. What We're Building (one paragraph)

A personal **web app (PWA)** that connects to your WHOOP, learns *your* body's specific patterns through a per-user ML model, and turns your recovery score into a **bank-account-style ledger** with itemized receipts, a 3-day forecast, a what-if simulator, and — most importantly — an **inverse planner** that can solve in reverse from a recovery target back to the behaviors required to hit it. It's *not* a chatbot. It's a math model of one specific person's body. That's the whole point.

---

2. The Strategic Insight — Why This Stands Out

WHOOP Coach (the GPT chatbot inside the WHOOP app) already answers "*why is my recovery low today?*" So we're **not** competing on that. Coach has a fixed architecture — it's a generic LLM, locked to WHOOP's data silo, prose-only, reactive Q&A, with no per-user model. That architecture has **structural blind spots no amount of prompt engineering can fix**.

We exploit those blind spots. Recovery Debt builds **three capability classes** on the same personal Ridge backbone — Coach has zero of them:

#	Capability class	What it means	Coach can't because…
1	**Forward prediction with numeric attribution**	"–10 sleep, –6 alcohol, –2 meeting → 58"	Coach gives prose, not a numeric receipt
2	**Backward attribution & counterfactual replay**	"Last Tuesday with 8h sleep instead of 5h, your week looks like…"	Coach has no executable model of your body
3	**Inverse planning**	"Target 75 by Saturday → sleep 9h Wed-Fri, no alcohol Thu-Fri, strain ≤12 Fri"	Inverse problems require gradient access to a parametric model. Coach has language, not math.

Three different capability classes on one model. **That's the senior interview framing.**

---

3. The One-Liner Comparison

You ask…	WHOOP Coach	Recovery Debt
"Why is my recovery low?"	A paragraph of plausible reasons	Itemized receipt: −10 sleep, −6 alcohol, −2 meeting
"How do I prepare for race day?"	General advice ("sleep well")	A solved plan: required sleep/strain/alcohol values
"How sensitive am I to alcohol?"	Generic average across all members	Your specific Ridge coefficient, with stability whiskers
"What did this year cost me?"	Can't answer	Cumulative SHAP wallet: alcohol cost you 412 pts, sleep saved 680
"What if I'd cut drinks last Friday?"	Speculation	Counterfactual replay with quantified ripple effect
"Tomorrow's calendar is brutal — how should I prep?"	Doesn't see your calendar	Reads it; suggests the smallest behavioral lever

---

4. The Six Features — What We're Actually Putting In

Tier 1 — Ship in the 3-week build (these define the project)

★ A. Inverse Recovery Planner ("Solve for Saturday")

What it does: User types a recovery target ("75 by Saturday for race day"). The app returns a concrete plan: required sleep, strain, and lifestyle values. When the target is infeasible, it returns the closest achievable recovery and which constraint hit its bound.

Why Coach can't: Coach is forward-only. Inverse problems require constrained optimization on a parametric model. Coach has neither.

How: `scipy.optimize.minimize` with SLSQP on the Ridge model. Minimize $\|x - x_baseline\|^2$ subject to $\beta \cdot x \geq y*$ and physiological feasibility bounds (sleep ∈ 5–10h, strain ∈ 0–21, alcohol ≥ 0).

Effort: 3–4 days. **Recruiter signal:** Very high.

★ B. Personal Sensitivity Profile ("Body Personality" page)

What it does: A one-page bar chart showing how *your* body uniquely responds to each input — sleep, alcohol, stress, etc. — with stability whiskers. "*Your sleep sensitivity: 4.2 pts/hour (stable). Your stress sensitivity: 1.1 pts/level (still learning).*"

Why Coach can't: Coach has no per-user model. The phrase "*your* alcohol sensitivity is X" is **only** expressible if a personal parametric model exists.

How: After ~30 nightly retraining runs, aggregate the median + IQR of each Ridge coefficient across versions. Render as horizontal bar chart with error whiskers.

Effort: 1 day. **Recruiter signal:** Very high. (Clearest one-image proof of personalization.)

★ C. Cumulative SHAP Wallet ("Year-in-Review")

What it does: An area chart of cumulative SHAP contribution by category over months. "*Days you logged alcohol cost you 412 pts this year. Sleep consistency saved 680. Strain on hard training days cost 287.*"

Why Coach can't: Coach builds every reply from scratch from a per-query context window. It has no longitudinal SHAP corpus and no concept of aggregable attribution.

How: SQL aggregation over the `shap_values` table grouped by feature category. Re-explain historical days through the *current* model nightly so values are comparable. Recharts area chart.

Effort: 1.5 days. **Recruiter signal:** High. (Spotify-Wrapped energy in a Loom demo.)

Tier 2 — Stretch (post-launch / v2; mention in README)

D. Counterfactual Replay

What: Mutate one feature on a past day, re-run the feature pipeline forward through the rolling-window cascade. "*Last Tuesday with 8h sleep, here's how the next 7 days would have played out.*"

Why Coach can't: Coach can speculate in prose; it cannot quantify the ripple effect through `strain_3day_avg` and `sleep_debt_3day` because it has no executable per-user model.

Critical design note for v1: Build `build_feature_matrix` as a **pure function** so this becomes a 2-day add later.

E. Calendar-Aware Proactive Forecasting

What: App reads tomorrow's calendar, predicts the cost of the day you've already scheduled, suggests the smallest behavioral lever. Push notification at 9 PM: "*Tomorrow's a 4-hour meeting day. Forecast 54. Sleep 8h instead of 7 → forecast 63.*"

Why Coach can't: No Google Calendar OAuth scope, no future-event ingestion, no proactive notifications outside the WHOOP app.

F. Multi-Source Fusion with Provenance Layer

What: Tag every feature with its source (`whoop / strava / calendar / weather / checkin`). Every receipt gets a provenance badge: "*73% of today's explanation came from WHOOP, 18% from your check-in, 9% from weather.*"

Why Coach can't: Locked into WHOOP's silo by both product policy and architecture.

What We Deliberately Did NOT Add (and why)

Idea	Why we skipped
Voice journaling	UX add, not a structural moat
Friend / social accountability	Orthogonal to the AI-engineering recruiter signal
Privacy-first ONNX local inference	No real privacy story when data already lives on your server
Symptom correlations / medical	Medical-claim risk for a 3-week portfolio
Athletic periodization (TrainingPeaks)	Drifts from the explainability core
Beating Coach at being Coach	Don't try. Compete on what Coach can't do, not what it can.

---

5. The Complete Tech Stack

Layer	Tool	Why
Frontend framework	Next.js 14 + TypeScript	Industry standard, native to Vercel
Styling	Tailwind CSS + shadcn/ui	No CSS files, drop-in components
Charts	Recharts	Composable React charts
PWA	next-pwa	Adds manifest + service worker
Frontend hosting	Vercel	Free, one-command deploy
Backend	FastAPI (Python)	Plays with sklearn / pandas natively
Backend hosting	Railway	Cheap, supports cron jobs
Database + Auth	Supabase (Postgres)	DB + auth + storage in one product
ML model	scikit-learn — Ridge Regression	Linear → SHAP is *exact*, coefficients interpretable
Validation	TimeSeriesSplit + RidgeCV	NEVER random KFold on time-series
Explanations	shap.LinearExplainer	Exact, fast, cheap on Ridge
Optimization (Inverse Planner)	scipy.optimize (SLSQP)	Convex QP on a linear model
LLM (stretch)	Anthropic Claude API	Strict-JSON parser, not a chatbot
Push notifications	Web Push API + VAPID	No third-party dependency

What we deliberately don't use: MongoDB, Redis, Docker, Kubernetes, Kafka, separate auth provider — all overkill for one user.

---

6. The 3-Week Plan (compressed)

Week 1 — Plumbing 🔧

Goal: open the app, log in, see 6 months of recovery as a line chart.

Day	Build
1	Next.js + FastAPI + Supabase project skeletons
2	WHOOP OAuth + DB schema (10 tables)
3	Backfill 6 months of WHOOP data
4	Webhooks + 4 AM safety-net cron
5	First chart on the dashboard + early Vercel/Railway deploy

Week 2 — Brain 🧠

Goal: open the app, see "*Your recovery dropped 18 today: sleep −10, alcohol −6, late meeting −2.*"

Day	Build
6	15-second daily check-in form
7	Feature engineering — `build_feature_matrix` (PURE function)
8	Train Ridge with TimeSeriesSplit + RidgeCV
9	Add SHAP LinearExplainer + integrity unit test
10	Nightly cron + receipt UI on dashboard

Week 3 — Standout Features 🏆

Goal: ship the 3 things WHOOP Coach physically cannot do, then go live.

Day	Build
11	Banking-style ledger UI with running balance + 3-day forecast
12	What-if simulator (sliders, live cascade)
13	**Inverse Planner** — `scipy.optimize` with SLSQP ★
14	Sensitivity Profile + Cumulative SHAP Wallet ★
15	PWA + demo mode + final deploy + 60–90 sec Loom

---

7. The Honesty Rules (the thing recruiters care about)

These come straight from the PRD §13 — break them and a sharp recruiter will dock you. Coach overclaims; we don't. **Honesty is a feature, not a flaw.**

❌ Don't say	✅ Say instead
"Alcohol costs you 11 points."	"On days you logged alcohol, your model predicted 11 points lower."
"You should sleep more."	"Days with longer sleep had higher predicted recovery."
"Alcohol is bad for your heart."	(Never make medical claims. Ever.)

Plus: **before day 60**, every insight is labeled "early estimate" with a confidence interval.

---

8. The 90-Second Recruiter Demo (what they see)

When someone clicks your live link, they should be able to do all 7 things in under 90 seconds without explanation:

1. ✅ See your live recovery chart (Week 1 work)
2. ✅ Open today's "receipt" — SHAP breakdown with real numbers (Week 2)
3. ✅ Drag a slider in the what-if simulator and watch the cascade (Week 3 Day 12)
4. ✅ Type a target into the **Inverse Planner** and see a real plan come back (Week 3 Day 13) — *the hero moment*
5. ✅ Open the **Sensitivity Profile** and see "*this user is more sensitive to alcohol than to stress*" (Day 14)
6. ✅ Open the **Cumulative SHAP Wallet** and see "*alcohol cost 412 pts this year*" (Day 14)
7. ✅ Install the PWA on their phone (Day 15)

Side-by-side comparison shot for the README: WHOOP Coach's prose explanation next to Recovery Debt's SHAP receipt for the same day. The visual contrast is the strongest single-frame proof of differentiation.

---

9. The Elevator Pitch (memorize this)

> *"WHOOP Coach is a generic LLM with the user's data piped into the prompt — it's reactive Q&A and gives prose, not numbers. Recovery Debt is a different paradigm: a per-user Ridge model with SHAP attribution that does three things Coach architecturally can't — forward prediction with exact numeric receipts, backward attribution aggregated into a year-in-review, and inverse planning where you give the app a recovery target and it solves for the behaviors required to hit it. The interesting engineering wasn't beating Coach; it was building the personal-model + explainability + optimization stack honestly, including the cold-start problem and the correlation-vs-causation gap."*

That paragraph turns the "WHOOP Coach exists" objection from a threat into a structured technical conversation.

---

10. What This Project Demonstrates (interview talking points)

A senior-grade portfolio piece in 3 weeks:

1. **Real ML pipeline** — feature engineering with lagged + rolling-window features, time-series CV, nightly retraining loop
2. **Model explainability** — SHAP with an integrity unit test
3. **Constrained optimization** — `scipy.optimize` SLSQP on top of a trained model (the Inverse Planner)
4. **Coefficient stability analysis** — IQR across model versions = production model-monitoring concept
5. **Longitudinal attribution aggregation** — SHAP as an aggregable signal, not just per-prediction
6. **Full-stack delivery** — OAuth 2.0, webhooks, signature verification, scheduled jobs, PWA, public deploy
7. **LLM as a tool, not the product** — strict-JSON parser (stretch), not a chatbot
8. **Honest framing of model limits** — confidence intervals, "early estimate" labels, correlation-vs-causation discipline
9. **Testing discipline** — purity tests, SHAP integrity tests, time-series leakage tests
10. **Synthetic-data engineering** — preserving realistic covariance for demo mode

---

11. Where Everything Lives (file map)

```
~/Desktop/Projects/recovery-debt/
├── OVERVIEW.md                  ← this file (master summary)
├── OVERVIEW.pdf                 ← printable version
├── PLAN.md                      ← daily checklist (simple terms)
├── BUILD_GUIDE.md               ← technical build doc (commands + code skeletons)
├── BUILD_GUIDE.pdf              ← printable version (34 pages)
├── Recovery_Debt_PRD.md         ← full design spec (18 sections, 412 lines)
└── docs/                        ← original source materials
    ├── Recovery_Debt_Project_Plan.pdf
    ├── Recovery_Debt_Data_Model_and_ML.pdf
    └── transcripts/
        ├── bank_account.txt
        └── itemized_receipt.txt
```

How to use them together:

Document	When to open it
`OVERVIEW.md`	When you need to remember the whole story in 5 minutes
`PLAN.md`	Daily — your todo list for today
`BUILD_GUIDE.md`	When you sit down to code (commands, snippets, file paths)
`Recovery_Debt_PRD.md`	When you need to remember *why* a design decision was made
`docs/*`	Original source material from WHOOP/research

---

12. The Bottom Line

You don't have to beat WHOOP Coach at being WHOOP Coach. **You can't.** Coach has in-app distribution, WHOOP's data team, and a head start.

But you can build the **opposite paradigm**: numeric instead of prose, personal instead of generic, predictive instead of reactive, multi-source instead of siloed, honest instead of overclaiming.

That's not a competitor — that's a different product. And the moment a recruiter sees your live demo type "*target 75 by Saturday*" into your inverse planner and watch a concrete plan materialize, they know they're looking at someone who **builds models**, not someone who calls APIs.

That's the win.

Now go build it. Day 1 starts with one command:

```
mkdir recovery-debt && cd recovery-debt && git init
```